Approximation methods for TSP path length on road networks

Koen Stevens

June 12, 2025

Bachelor's Thesis Econometrics

Supervisor: dr. N.D. van Foreest

Second assessor: dr. W. Zhu

Approximation methods for TSP path length on road networks

Koen Stevens

Abstract

This study investigates the applicability of the Beardwood-Halton-Hammersley (BHH) formula for estimating Traveling Salesman Problem (TSP) path lengths in real-world delivery scenarios using OpenStreetMap data from various Dutch neighborhoods. While the BHH formula yields accurate predictions with a Mean Absolute Percentage Error (MAPE) of 6.6% when the area-specific proportionality constant β is known, its performance degrades significantly (with MAPE exceeding 22%) when using a nationwide average β. This variability limits its practical utility, since this formula is mainly used in applications where the true value of β is not known. Even with area-specific estimates of β, accuracy varies depending on address distribution and road network connectivity. To overcome these limitations, a supervised learning model leveraging geographic and spatial features was developed, achieving improved generalizability and a lower MAPE of 10.6% on the test set. However, prediction errors remain high in areas with clustered or irregular delivery patterns.

1 Introduction

The Traveling Salesman Problem (TSP) is an important problem in operations research. It is relevant for last-mile carriers and other logistics companies where efficient routing directly impacts cost, time and service quality. Since the number of parcels worldwide has increased between 2013 and 2022 and is expected to keep increasing (Statista, 2025), the need for fast, scalable route planning methods becomes ever more pressing.

The TSP is an NP-hard problem, it is computationally intensive to find the exact solution for large instances. In many real-world scenarios, the exact optimal routes may not be needed, but instead a rough, reliable estimate of the optimal route length. For instance, consider a last-mile carrier expanding to a new area for service. This firm may need to hire new delivery persons. Then, this firm needs to estimate how many delivery persons they need to hire, and what kinds of contracts they should provide. Reliable estimates for the route length can provide valuable information for making such decisions.

Efficient approximation methods provide a solution for such practical applications where exact solutions are too computationally intensive to conduct or not feasible due to insufficient data. These methods aim at approximating the expected optimal total travel time or distance, while using minimal data and computational effort.

There is extensive research on such approximation methods and how they perform in the Euclidean plane. Consider n uniformly drawn locations from some area in $\mathbb{R}^2$ with area A. Beardwood, Halton, and Hammersley (1959) introduce the following theorem:

$$\frac{L}{\sqrt{n}} \to \beta\sqrt{A}, \quad \text{as } n \to \infty \tag{1}$$

as an estimation for the length of the shortest TSP path measured by Euclidean distance through these random locations, where β is some proportionality constant.

In contrast to the Euclidean plane, TSP path lengths in an area might behave differently based on different features of this area,. For example, clusters of locations with empty space in between them or natural boundaries such as canals could have impact on TSP path length in an area. The BHH formula is not able to capture such complex relations, and might fail to provide accurate results.

This research investigates how well Equation (1) performs, predicting TSP path length on real road networks. Using OpenStreetMap (OpenStreetMap contributors, 2025) data, we simulate TSP instances in a variety of different urban areas in the Netherlands, then solve these for the actual shortest paths using the Lin–Kernighan heuristic (Lin and Kernighan, 1973). Next, the β from Equation (1) is estimated and the performance of this formula is analyzed. Additionally, the results for β and the performance across the selected areas is compared. In an attempt to capture more complex relations between area features and TSP path length, a supervised learning method is used as an alternative model to predict TSP paths.

The core contributions of this research are:

1. An analysis of the performance of the BHH formula under the following conditions:
   (a) The locations are drawn from the set of postal codes in the area in question, a violation of the uniform distribution of visited locations. In reality, delivery locations for companies are not uniformly distributed across the delivery area, they may be clustered in certain parts of the area.
   (b) Applied to realistically sized (in the sense that a delivery person could serve this area in a single workday) real-world parts of cities and villages in the Netherlands;
2. A supervised learning analysis to investigate how well the optimal TSP path length can be predicted, based on features of the area, including road network density, address density and other natural and man-made features of the area.

The analysis can easily be extended to any type of area in any part of the world, one would only have to download the OpenStreetMap (OpenStreetMap contributors, 2025) for another part of the world and add the names of the areas to apply it to. The source code of this project is available on GitHub.

In section 2 the context and previous research in this field is provided. Section 3 provides a formal problem definition of the problem. Section 4 contains a description of the data that is used, and explains how it is processed. In section 5 the design of the experiment is documented, section 6 displays the results. Section 7 concludes the research.

2 Literature Review

In this section the existing literature on the BHH formula and some applications, and on the Lin-Kernighan heuristic and its implementations is reviewed.

2.1 Applications of the BHH formula

This research concerns the performance of Equation (1) for reasonable amounts of locations n a delivery person can visit in a workday, say $20 \leq n \leq 90$. Lei, Laporte, Liu, and

Zhang (2015) estimates the values of β for a selection of values for n. In their research, the points were generated uniformly across the area and the L_2 distance metric was used. Table 1 lists the results.

Table 1: Empirical estimates of β as a function of n, $20 \leq n \leq 90$ (Lei et al., 2015)

n	$\beta(n)$
20	0.8584265
30	0.8269698
40	0.8129900
50	0.7994125
60	0.7908632
70	0.7817751
80	0.7775367
90	0.7773827

Figliozzi (2008) is the first research to apply approximation formulas to real-world instances of TSPs (and VRPs (Vehicle Routing Problems)), in the sense that distances between locations are measured over the actual road network. An extension of Equation (1) that works for VRPs is assessed in this setting. It is found that this model has an R^2 of 0.99 (which means 0.99 of the variation in TSP path length is explained by the formula) and MAPE (Mean Absolute Prediction Error) of 4.2%. This prediction error is slightly higher than when it is applied to a setting where Euclidean distances are considered (3.0%) (Figliozzi, 2008).

Merchán and Winkenbach (2019) use circuity factors to measure the relative detour incurred for traveling in a road network, compared to the Euclidean distance. This circuity factor for locations p and q is defined as:

$$c = \frac{d_c(p,q)}{d_{L_2}(p,q)} \tag{2}$$

By definition, $c \geq 1$, as the Euclidean distance is the shortest possible path between two points in a plane. Then, β_c is estimated by $\beta_c = c\beta_{L_2}$, where β_{L_2} is estimated by minimizing the errors of Equation (1) using the L_2 distance metric. This value c, is estimated for three different areas in São Paulo, for which the results are listed in Table 2. These values indicate real travel distances are on average 2.76 times longer in area 1 compared to the L_2 metric. The values are obtained by generating n locations (for n ranging from 3 to 250) uniformly across the area, computing near-optimal tour lengths under the L_2 metric, solving for β_{L_2} by minimizing the errors of Equation (1) and then scaling by the circuity factor.

Table 2: Estimates of the circuity factor c and its corresponding β_c (Merchán and Winkenbach, 2019)

	Area 1	Area 2	Area 3
c	2.76	2.34	1.82
β_c	2.48	2.10	1.64
MAPE (%)	12.65	8.72	10.62

Their model is able to predict TSP path length more accurately compared to when using the L_1 distance metric, predicting TSP path length using the L_1 distance metric they find MAPEs between 24% and 58%.

It is important to note that the assumptions in Merchán and Winkenbach (2019) may limit the generality of the findings. In particular, the use of locations uniformly distributed across the area does not accurately reflect the spatial distribution of delivery points in real urban environments, where locations tend to cluster. Within urban areas, large apartment buildings and single-family homes may coexist in the same neighborhoods, which implies that addresses and thus delivery locations are not distributed uniformly across the area in reality. The usage of uniformly generated delivery locations might imply that the prediction errors found in their research are underestimating the true prediction errors when their models are applied to real-world scenarios.

2.2 Lin-Kernighan Heuristic

To be able to efficiently solve many TSPs, to find a good estimate for β, a fast and reliable solution algorithm is needed. The Lin-Kernighan heuristic (Lin and Kernighan, 1973) is generally considered to be one of the most effective methods of generating (near) optimal solutions for the TSP. In this research a modified implementation of the heuristic is used (Helsgaun, 2000). The run times of both heuristics increase by approximately $n^{2.2}$, but the modified heuristic is much more effective. It is able to find optimal solutions to large instances in reasonable times (Helsgaun, 2000).

3 Problem definition

This section provides a formal definition of the problem this research aims to solve. Define N_j to be the set of all postal codes in a neighborhood j. Let $TSP_{n,j} = \{x_i\}_{i=1}^{n}$, $x_i \in N_j$ be a subset of size n of the postal codes in neighborhood j. Then, $L_{n,j}$ can be defined to be the length of the shortest path through the set $TSP_{n,j}$. This is the length of the solution of the traveling salesman problem consisting of the points in $TSP_{j,n}$. We first want to examine the behavior of $TSP_{j,n}$, and most importantly the accuracy of the BHH formula in estimating this value. Afterward, we want to see how using supervised learning can be used to get a more accurate estimate of $TSP_{j,n}$. This research consists of two parts.

1. Analyze the behavior of $L_{n,j}$ for various sample sizes n and neighborhoods j, in the context of evaluating the accuracy of predictions made by Equation (1).

2. Investigate how well TSP path length can be predicted using n and a number of features of the neighborhood the TSP is in, using supervised learning.

4 Data

In this section, a detailed explanation of the data is provided. This includes the characteristics of the data used and how this data is processed.

4.1 Source

In order to model the complex nature of real road networks, data from OpenStreetMap (OpenStreetMap contributors, 2025) is used. OpenStreetMap is an open-source project that provides geographic data, including accurate and detailed information about roads, buildings and natural features around the world. The data is continuously maintained and updated by a large community of volunteers, making it a valuable resource for this research.

This data can be downloaded from Geofabrik, and then exported to a `PostgreSQL` (PostgreSQL Global Development Group, 2025) database using `osm2pgsql` (Burgess and Contributors, 2025), in order to be able to efficiently use the data with `Python`.

4.2 Description

For this analysis, the database has three interesting tables: `planet_osm_polygon`, `planet_osm_nodes` and `planet_osm_ways`. Many neighborhoods in the Netherlands have a polygon defined in the data. In OpenStreetMap a polygon is a closed shape formed by a set of geographic coordinates (`nodes`) that are connected by lines (`ways`). These objects can be used to define boundaries of geographic areas, such as lakes, parks, nature reserves and parts of cities and villages. In this research the polygons are used to efficiently filter the buildings and roads only in a certain area. These polygons are stored in `planet_osm_polygon`.

In the database, the roads are defined as `ways`. A way has three attributes: `id`, `nodes` and `tags`. The attribute `nodes` contains an ordered list of the nodes that this road consists of. In the `tags`, information about the way is stored, for instance whether it is a one-way road, or the type of road (for example primary or trunk). The information about the roads that are needed for this analysis is the road ID, the IDs of the nodes the road consists of, the coordinates (Latitude, Longitude) of these nodes, and whether the road is a one-way road.

The buildings are stored as `nodes`. In this table (`planet_osm_nodes`), a large amount of other objects are stored as well. For this analysis the potential delivery locations need to be extracted. Some of these nodes have a postal code defined, which can be used to extract all buildings that a potential delivery could take place. This way, for example a little shed in someone's backyard is also filtered out, since this does not have its own postal code. For this research only the node ID and the coordinates are needed.

In order to perform the supervised learning analysis, the second contribution of this research, features of the neighborhoods need to be extracted. In the `planet_osm_ways` table, not only roads are stored, there is also a large amount of other objects stored in here, such as parks, bodies of water, what a certain area is used for (i.e. residential, commercial) and other natural or man-made objects. Features like this might provide predictive value for TSP path length.

4.3 Graph construction

Using the geographic information of the roads and buildings, a graph is constructed for each neighborhood, using the `igraph` (igraph, 2025) module in `Python`. This graph connects all buildings to each other over the road network. First, using the information about the roads and buildings the sets of nodes and edges need to be defined. An edge is a line that connects two nodes. Extracting the edges connecting the road networks

is straightforward, since all roads already have an ordered list of nodes defined. The subsequent nodes simply need to be stored as pairs, and all edges are defined.

When the road network is defined as a set of nodes and edges, the next step is to connect the buildings to the road network. This needs to be implemented manually, since no data is stored in OpenStreetMap about to which road the buildings belong. This algorithm needs to be very efficient, since many buildings are added in each area. An R-tree can be used to accomplish this efficiency. An R-tree is a dynamic index structure that is able to retrieve data items quickly according to their spatial locations (Guttman, 1984). Listing 1 displays the algorithm used to make the edges that connect the buildings to the road network. For each building node, the closest point on the closest road is found, using the shapely (Shapely, 2025) implementation of the R-tree, **`STRtree`**. Then, if this point is not an already existing node, a new virtual node is added, which splits this existing edge in two parts. The road is reconnected with the new node in between. Finally, the building is connected to this new node.

In order to find the shortest path in terms of real distance, and to calculate the length of the TSP path, a 'weight' needs to be added. This weight represents the length of this edge in meters. The equirectangular approximation is used to calculate these weights. This approximation is very efficient, but only works when the points the distance is calculated between is small enough such that the rounding of the earth does not have a significant effect on the true distance. The nodes are all close enough together, so the effect of the rounding of Earth's surface is negligible. Let A and B be two nodes, and (φ_A, θ_A), (φ_B, θ_B) be their coordinates, in `radians`. Let $R = 6,371,000$ meters, Earth's radius. Then the distance between these nodes (the weight of the edge connecting them), using the equirectangular approximation is:

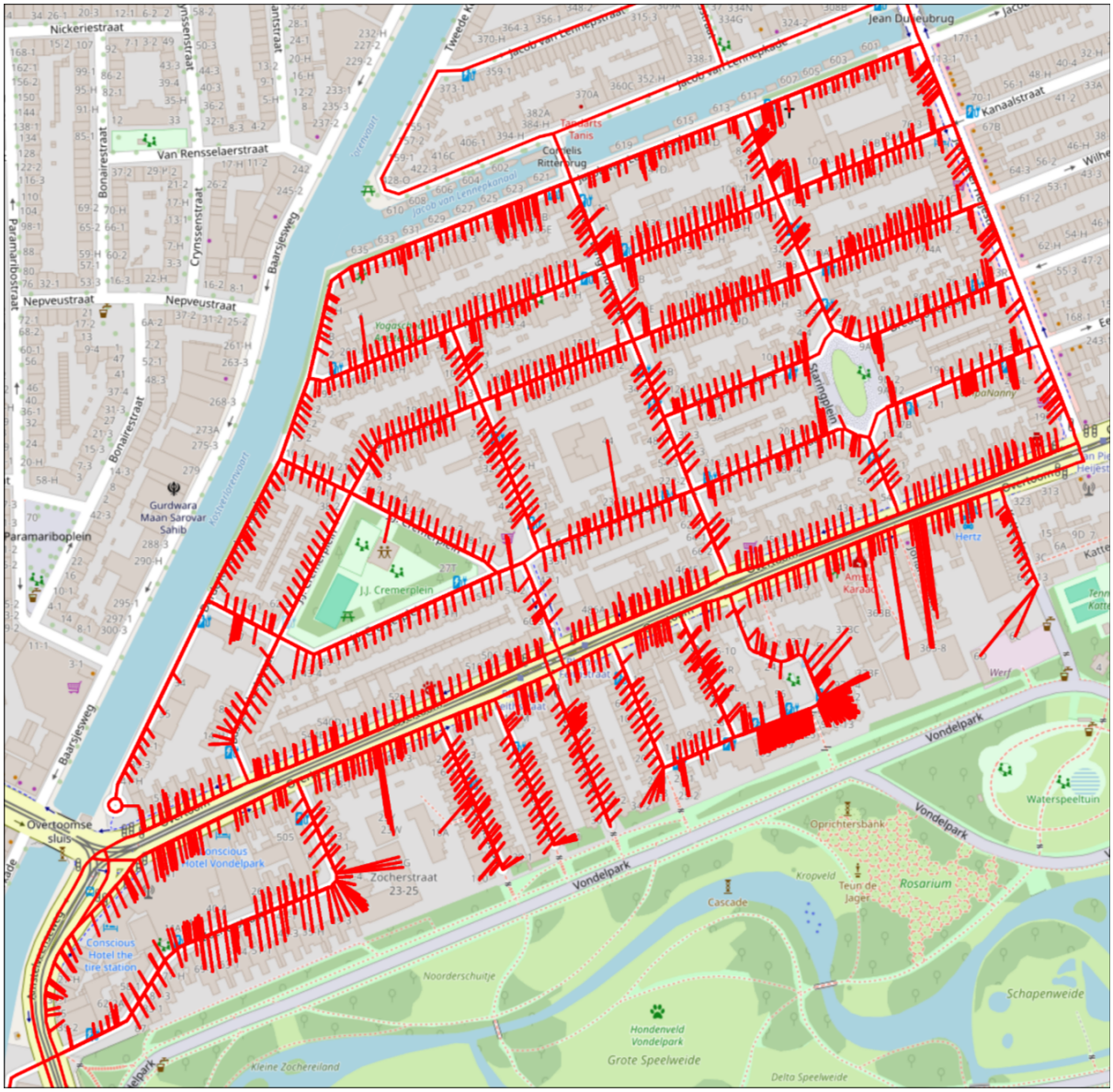
$$d(A, B) = R\sqrt{\varphi^2 + \theta^2}, \tag{3}$$

$$\text{where } \varphi = (\varphi_B - \varphi_A)\cos\left[\frac{\theta_A + \theta_B}{2}\right], \tag{4}$$

$$\text{and } \theta = \theta_B - \theta_A. \tag{5}$$

Using the `folium` module in `Python`, such a graph can be visualized on a geographic map. One of such visualizations is provided in Figure 1. All maps like this one, for the areas in this analysis can be viewed and interacted with via this `webpage`.

Figure 1: Visualization of the graph, for the Overtoomse Sluis quarter in Amsterdam.



4.4 Extracting features

One of the predictors of the TSP path length, is the area of the neighborhood the locations are drawn from. The OSM data provides the area of the polygons, but this value can not be used to predict TSP path length effectively. This value significantly overestimates the correct value for A when there are things like parks, lakes, or large buildings are inside in the quarter. These places have no delivery locations but do increase the area of the quarter. Consequently, this area should not be included in the area calculation. To account for this problem, alpha shapes are used to calculate A. Alpha shapes are generalizations of the convex hull, they capture the shape of a point set (Akkiraju, Edelsbrunner, Facello, Fu, ücke, and Varela, 1995). Figure Figure 2 displays the map of the Beijum quarter in Groningen with the alpha shape for its addresses, as it was used for calculating the area. It can be seen that using this method, large areas without buildings, such as the green-space in the middle is being left out, as desired.

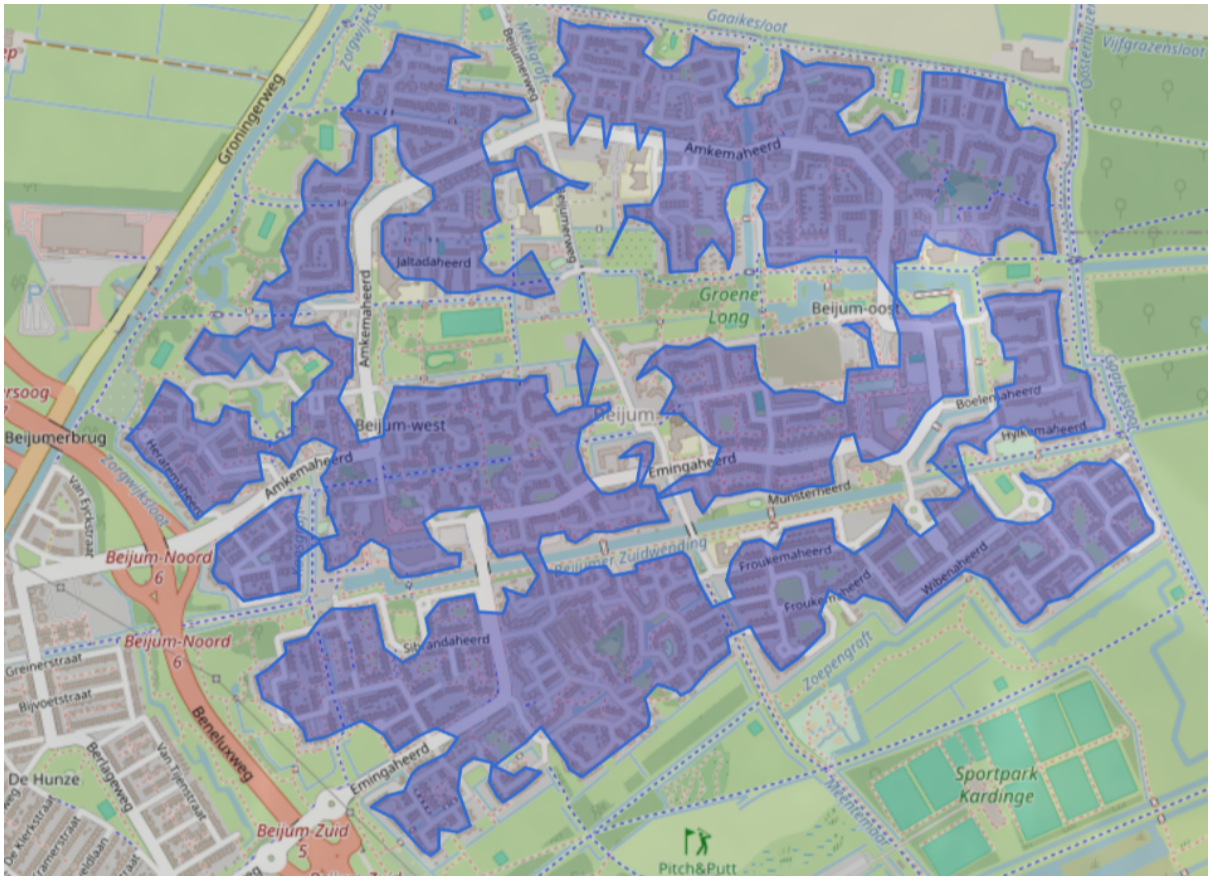


Figure 2: The map of the Beijum quarter in Groningen, with the alpha shape around its addresses.

While the area of a neighborhood is an important predictor, it is only one of many features considered in the supervised learning model in the second part of this research. To train a supervised learning model that predicts the TSP path length based on neighborhood characteristics, a comprehensive set of features needs to be extracted. In total, approximately 80 features are gathered for each neighborhood. These features describe various aspects of the built environment, infrastructure, and spatial distribution of delivery locations. The features fall broadly into the following categories:

- **Geographic features:** the area of the alpha shape around all postcodes, characteristics such as how much water or how many parks, and land use (i.e. residential or commercial).
- **Road network metrics:** Percentage of roads that are one-way, total length of road network divided by the area of the alpha shape.
- **Graph-technical properties**: average edge length, average path length between all nodes in the graph, clustering properties.

The clustering properties might be particularly interesting. The assumption in the BHH formula that the locations are uniformly distributed across the area is the one the setup in this research violates. The assumption is violated on two levels: there might be different sub-neighborhoods, of different size and address density in a quarter, with space in between them. Additionally, on a smaller scale, a neighborhood might contain mostly

detached houses or semi-detached houses, with a few larger apartment buildings. Then, the assumption of uniformly distributed locations is also violated. Using these properties in the supervised learning model might lead to more accurate predictions. The features that were used for this are:

- The number of communities according to the `infomap` method, divided by the area of the alpha shape (Rosvall and Bergstrom, 2008). This might provide information about clustering of addresses on the scale of sub-neighborhoods inside a quarter with space in between.
- The mean, max, and variance of the degree of the nodes. The degree of a node is the number of edges are connected to it. For example, when there is a large apartment building, often times all these different addresses connect to the exact same spot on a road. Then, this road node has a very large degree, sometimes more than 100. The mean, variance and maximum of this number all might provide some information about this local clustering.
- The diameter of the graph: the maximum shortest-path distance between any two nodes.
- The radius of the graph: the minimum eccentricity of any node, where eccentricity is the maximum shortest-path distance to any other node.

5 Experimental design

This section provides an overview of the methods and algorithms used to solve these problems.

5.1 Evaluation of BHH formula

The following algorithm is used for the first part of the research. Let $L = \{L_i\}$ and $\hat{L} = \{\hat{L}_i\}$ be the lists of observed and predicted TSP tour lengths, respectively, over all sample sizes j and repetitions k for neighborhood n. Let $|L|$ denote the total number of evaluated instances per neighborhood.

Algorithm 1 Procedure for evaluating the predictive accuracy of Equation 1

for each neighborhood j **do**
  Extract features of this neighborhood and save to a file for later use
  Construct a graph of the road network and visualize on a map
  Initialize empty lists for L, $\hat{L}$ and β
  for each sample size $n \in \{20, 22, \ldots, 86, 88\}$ **do**
    for each repetition $k \in \{0, 1, \ldots, 9\}$ **do**
      Randomly sample a subset $TSP_{n,j} \subset N_j$ of size n uniformly over the addresses
      Solve the TSP over $TSP_{n,j}$ to compute $L_{n,j}^{(k)}$
      Estimate $\beta_j^{(k)}$ by solving Equation (1) for β using $L_{n,j}^{(k)}$
      Append $L_{n,j}^{(k)}$ to list L, and $\beta_j^{(k)}$ to list β
  Compute $\hat{\beta} = \mathsf{E}[\beta]$
  Compute the predicted lengths $\hat{L}$ using Equation (1) with the estimated value $\hat{\beta}$
  Compute the Mean Absolute Error (MAE) for neighborhood j:

$$\text{MAE}_j = \frac{1}{|L|} \sum_{i=1}^{|L|} \left| L_i - \hat{L}_i \right|$$

  Compute the Mean Absolute Percentage Error (MAPE) for neighborhood j:

$$\text{MAPE}_j = \frac{100\%}{|L|} \sum_{i=1}^{|L|} \left| \frac{L_i - \hat{L}_i}{L_i} \right|$$

  Visualize the results in graph form, a scatter plot of TSP length against number of locations, with a fitted Equation (1) and a histogram of the errors $\epsilon_i = L_i - \hat{L}_i$.
  Visualize a random TSP solution on a map.
  Save the results for L to a file for later use.

5.2 Supervised learning method

In the second part of this research, a supervised learning model is trained to predict the TSP path length from neighborhood-level features and the number of points j. Specifically, a Random Forest regression model is employed.

The data set is constructed using the results from Algorithm 1. Each entry of L (the list of TSP path lengths) is a separate observation in this data set. For every observation, the corresponding area features and the number of locations j are stored in the data set. Then, the following steps are performed:

1. Split the data into a training and test set, 80/20 split.
2. Normalize the features such that they all have mean 0 and variance 1.
3. Train a Random Forest regressor to predict TSP path length L using the number of points j and neighborhood-level features as input variables.
4. Evaluate model performance on the test set using the metrics (for the test data):
   - Mean Absolute Error (MAE)
   - Mean Squared Error (MSE)

- Coefficient of determination R^2

5. Analyze feature importance to assess which neighborhood characteristics most influence the TSP length.

6. Re-run the model with a reduced feature set, only using the most important features. This requires some testing, to analyze what the treshold for dropping out features should be.

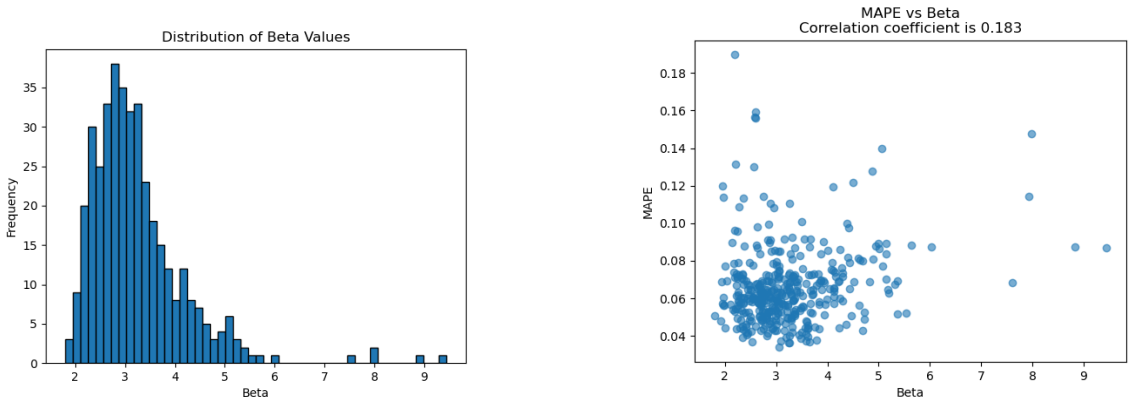
6 Results

In this section the results of the analysis described above is laid out. First, we analyze the performance of the BHH formula and the implications of these results, afterward we discuss the supervised learning method.

6.1 Evaluation of BHH formula

6.1.1 Main Finding and implications

The main finding is that the formula performs poorly and its core advantage (its simplicity) breaks down when realistic applications require estimating its parameters through costly simulations. The results for β, the MAE and MAPE using the BHH formula are tabulated in Table 5 (Appendix). The value for β differs strongly from area to area: some areas have an estimated β as low as 1.5, while others have β close to 6. The prediction accuracy differs vastly per neighborhood: the Mean Absolute Percentage Error is below 4% for some areas, and above 15% for others (using their individual β). A histogram of the values for β, and a scatter plot of the values for β against the corresponding MAPE is provided in Figure 3. The correlation coefficient of β and the MAPE is 0.18. This indicates that there is no real correlation between the MAPE and β, the difference in the prediction accuracy across the areas does not stem from a difference in the proportionality constant.



(a) Histogram of β.

(b) Stad van de Zon

Figure 3: Scatter plot of β against the MAPE.

It is found that fitting Equation (1) yields a Mean Absolute Percentage error of about 6.6% averaged over all observations, when letting β differ per neighborhood, as can be

seen in Table 3. When using the nationwide average as the value for β to predict TSP path length, the model performance drops significantly, with a MAPE of more than 22%.

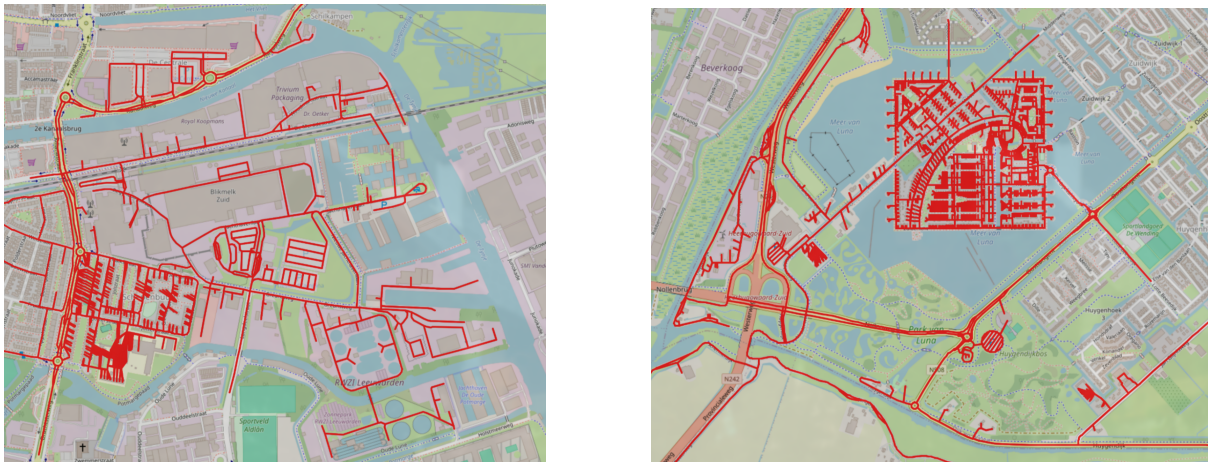
Table 3: Performance metrics for the BHH formula.

Model	R^2	MAE (m)	MAPE (%)
Varying β	0.95	957.75	6.59
Restricted β	0.30	3554.73	22.24

This suggests that the BHH formula performs poorly for estimating TSP path lengths on real-world road networks, especially for the number of stops a delivery person might make in a typical workday. Although the main appeal of the BHH formula lies in its simplicity and analytical form, it is not useful when the true value of β is unknown. An average prediction error of more than 22% much too high to use the formula in practice. Even when the true value of β is known, the prediction accuracy varies strongly across different areas. Moreover, estimating the true value of β requires simulations or calibration using known TSP solutions, undermining the original appeal of the BHH formula, its simplicity. Therefore, we conclude that the BHH formula is not suitable for practical applications where accuracy is critical. Instead, we recommend calculating the exact TSP solutions, which is computationally feasible using efficient algorithms such as LKH.

6.1.2 Potential explanations

Potential reasons for the variation in accuracy can be analyzed using visualizations. For example, the prediction error for the Schepenbuurt quarter in Friesland, and the Stad van de Zon quarter in Noord-Holland is especially high, compared to the other areas. These low-accuracy areas have an important feature in common. Figure 4 displays two of such areas.



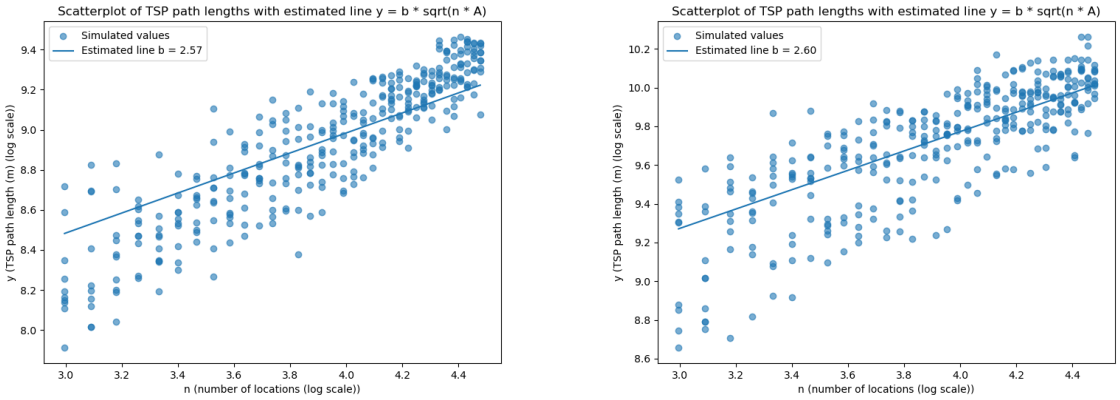
(a) Schepenbuurt (b) Stad van de Zon

Figure 4: Maps of Schepenbuurt and Stad van de Zon quarters.

Such quarters have one or more clusters of addresses, with some addresses spread out over the rest of the area, where the number of addresses in the cluster is significantly higher than the number of addresses spread out. A possible explanation for the higher

prediction errors is that when the number of locations to visit, j, is small and one of the clusters is significantly smaller than the other in terms of the number of addresses it contains, there is a substantial probability that none of the addresses from the smaller cluster are included in the TSP instance. This would result in a significantly lower path length, as the distance to the other cluster and back does not need to be traveled. However, there is also significant probability that there are addresses of the smaller cluster in the TSP, which in turn leads to a significantly higher path length. Based on this, a higher variation in the TSP path length is to be expected, which explains the higher prediction error. This problem disappears when the number of locations in the TSP j is large enough, since in that case locations from all clusters would be included.

This pattern is again confirmed by the visualizations in Figure 5. The Stad van de Zon scatter plot is the clearest example of this phenomenon.

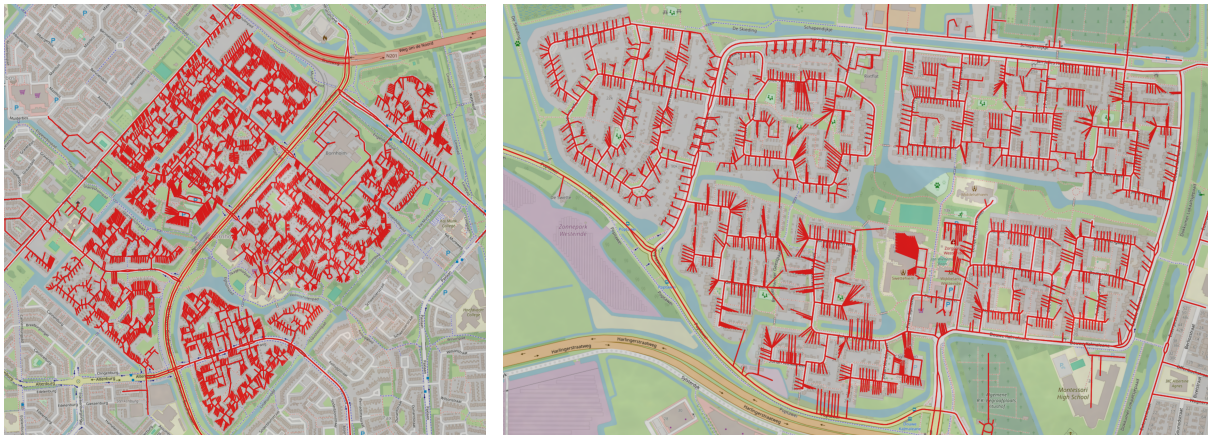


(a) Schepenbuurt (b) Stad van de Zon

Figure 5: Scatter plots of TSP path length vs number of locations for the quarters Schepenbuurt and Stad van de Zon (log-log scale).

There is a group of points below the fitted line for the BHH formula, and a group of points above it. The randomness in whether locations from outside the main cluster are included in a TSP decides whether its data point it in the upper or lower group of points. As n increases, the number of data points in the lower group decreases. Both of these observations suggest these data points are for a TSP that contains locations only in the main cluster, since the probability of this happening decreases with n.

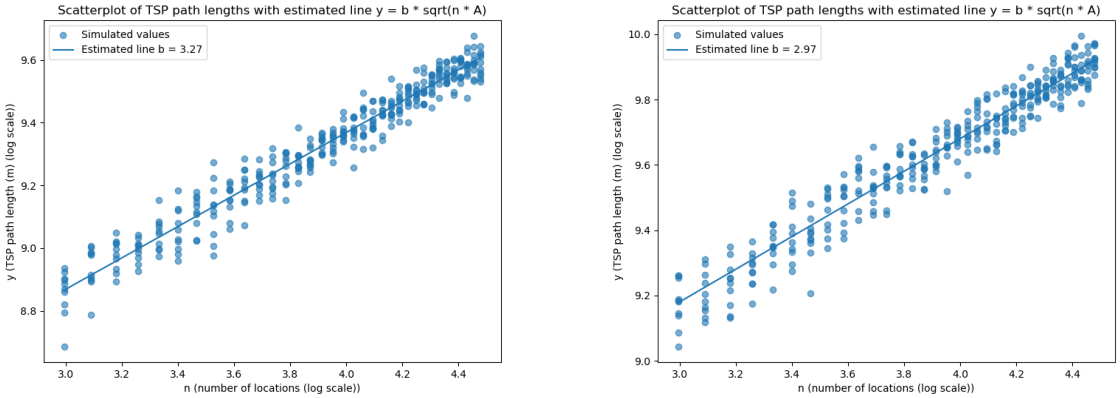
Additionally, it is interesting to consider neighborhoods where the prediction errors are especially low. Figure 6 displays maps for two quarters, Westeinde in Friesland and Bornholm in Noord-Holland. Such low-prediction error neighborhoods also share characteristics. One is that the addresses seem distributed close to uniformly across the area. There are some 'clusters', for example Westeinde can visually be split into four clusters that are only connected by one or two main road connections. However, they all contain a similar number of addresses, such that each TSP will likely visit all of them. This could explain why the formula gives lower prediction errors for these areas, compared to Schepenbuurt and Stad van de Zon.



(a) Westeinde (b) Bornholm

Figure 6: Maps of the quarters Westeinde and Bornholm.

Although some variation around the fitted BHH formula remains, Figure 7 exhibits significantly lower prediction error compared to Figure 5.



(a) Westeinde (b) Bornholm

Figure 7: Scatter plots of TSP path length vs number of locations for the quarters Westeinde and Bornholm (log-log scale).

6.2 Supervised learning method

A summary of the results of the Random Forest model is listed in Table 4. A grid search is used to find the hyper-parameters that provide the lowest out-of-sample prediction errors. This results in the selection of `max_depth`=10 and `n_estimators`=300 for the full model, and `max_depth`=20 and `n_estimators`=300 for the reduced model.

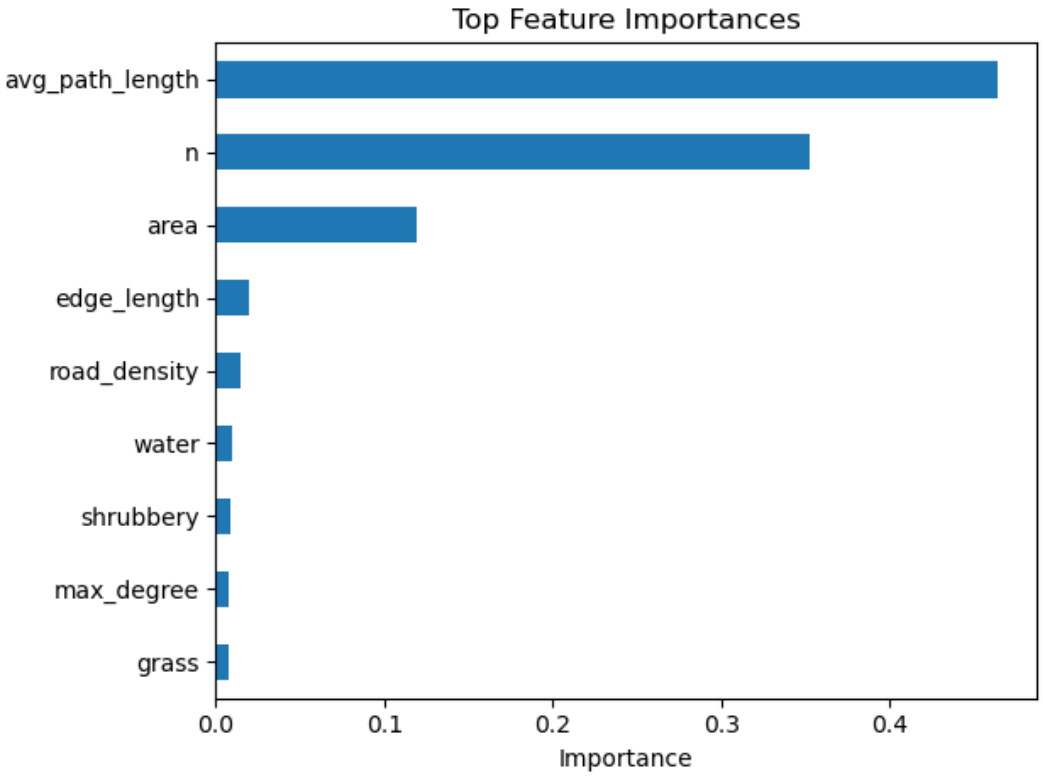
Table 4: Performance metrics for the Random forest models using all features and reduced features.

Model	R^2	MAE (m)	MAPE (%)
Full (train)	0.95	958.62	6.51
Full (test)	0.85	1631.59	10.86
Reduced (train)	0.96	831.17	5.62

Model	R^2	MAE (m)	MAPE (%)
Reduced (test)	0.85	1613.63	10.58

An R^2 of 0.96 is found on the train data for both the full and the reduced model, and an R^2 of 0.85 for both models on the test data. The MAPE is also nearly equal for both models. Figure 8 displays the most important features for predicting TSP path length, according to the Random Forest model.

Figure 8: The most important features in the reduced model.



These nine features are the only ones that are left in the reduced model. All other features get dropped out from the full model due to insufficient importance. The reduced model still provides equal prediction accuracy, requiring less data collection. The most important features are, according to the model:

- average path length between all nodes in the graph of the road network,
- the number of locations in the TSP n,
- the area of the neighborhood the TSP is in.

Interestingly, the most important feature according to the Random Forest, is a feature that does not appear in the BHH formula. It is obvious why the number of locations is an important predictor. The average path length as well as the area say something about how spread out an area is. The average edge length also carries some importance. This value might mostly imply something about how straight or curvy the roads are. A road with a lot of curves has many short edges, and a long straight road consists of a few long edges. The maximum value of the node degree also provides some predictive value. It is

difficult to interpret the importance of the features, since it might be that a feature that carries some importance does not directly affect TSP path length, but is correlated with some other variable that is not known which in turn has an effect on TSP path length.

Table 6 shows per-neighborhood Mean Absolute Percentage errors, for the test data, comparing the full and the reduced model. The model is able to predict TSP path length with below 5% for some areas, but for other areas the prediction errors are close to or above 30%. Again, similarly to the results for the BHH formula, the predictions for the Stad van de Zon quarter have the highest MAPE, of all areas. It seems that the Random Forest model is also not able to provide accurate results for areas with strong clustering of locations, as was the BHH formula, even though some features were included in the Random Forest model that could provide some information about this clustering. A potential reason for this might be that this clustering causes larger variation in the TSP path length, especially for smaller numbers of visited locations, which would imply that any approximation would yield higher prediction errors in such area.

Overall, the Random Forest model provides estimates with MAPE less than half of the MAPE of the predictions of the BHH formula (with estimated value for β). This is a strong improvement in performance, requiring little data collection to extend this to completely new areas.

7 Conclusion

This research evaluated the applicability of the Beardwood-Halton-Hammersley (BHH) formula to real-world delivery scenarios by analyzing its performance on TSP instances constructed from OpenStreetMap data across various Dutch neighborhoods. We find that the BHH formula provides predictions with Mean Absolute Percentage (MAPE) error of 6.6% on the TSP path length provided the true value β for the area is known. The proportionality constant β varies substantially between neighborhoods. As a result, when this value β is not known, and a nationwide average is used, this MAPE increases to more than 22% on average. The main appeal of the BHH formula is its simplicity and closed form. However, in many real-world applications the true value of β is unknown. Making predictions using the BHH formula would in such cases yield much too high prediction errors of more than 22% on average, which is not useful. Estimating the true value of β defeats the appeal of the BHH formula (its simplicity) since this requires simulations. Even when estimating a separate β per area, the prediction accuracy remains inconsistent. Neighborhoods with evenly distributed addresses and good connectivity tend to conform better to the BHH formula, while those with uneven-sized clusters show larger deviations in TSP path lengths.

To address these limitations of the BHH formula, a supervised leaning model is used to predict TSP path length based on area features about the spatial distribution and other geographic features. This model is able to provide more accurate predictions compared to the BHH formula with unknown true β, with a MAPE of 10.6% on the test set. This model offers a more scalable and generalizable approach for real-world applications, however it does not solve all limitations of the BHH formula. The predictions for areas with a clustered, irregular distribution of delivery locations still have a high MAPE. The importance of this for real-world applications is debatable, since a delivery company is likely to distribute their delivery areas in a way that decreases this irregularity.

We advise entities aiming to estimate TSP path length in real-world applications

for low numbers of stops to calculate the true TSP solutions. Using programs like the Lin-Kernighan heuristic, these calculations are very efficient and require little computing power, while gaining much prediction accuracy.

An important caveat for the models used in this research is that it is assumed delivery locations are on a flat, 2d plane. This is a reasonable assumption when considering Dutch neighborhoods, since the Netherlands does not contain much variation in terrain height. Terrain height does have a significant impact on the length of the shortest path between two locations, which this research omits from the list of features. For this reason it might not be valid to extend such approximation methods to areas with a large variation in terrain height.

Future research might be to extend the approximation methods for TSP path length to areas of varying terrain height. However, this might be difficult to apply, since no sufficient data about elevation of roads is included in OpenStreetMap. The data from OpenStreetMap would require to be combined with other data sets containing elevation information. Furthermore, approximation methods could also be applied to travel times instead of distances.

8 References

Akkiraju, N., H. Edelsbrunner, M. Facello, P. Fu, E. P. M ücke, and C. Varela (1995). Alpha shapes: definition and software. In *Proceedings of the 1st international computational geometry software workshop*, Volume 63, pp. 63–66.

Beardwood, J., J. H. Halton, and J. M. Hammersley (1959). The shortest path through many points. In *Mathematical proceedings of the Cambridge philosophical society*, Volume 55, pp. 299–327. Cambridge University Press.

Burgess, J. and Contributors (2025). osm2pgsql: OpenStreetMap data to PostgreSQL converter. Version 1.9.0, Accessed: 2025-04-25.

Figliozzi, M. A. (2008). Planning approximations to the average length of vehicle routing problems with varying customer demands and routing constraints. *Transportation Research Record 2089*(1), 1–8.

Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. pp. 47–57.

Helsgaun, K. (2000). An effective implementation of the lin–kernighan traveling salesman heuristic. *European journal of operational research 126*(1), 106–130.

igraph (2025). igraph - the network analysis package.

Lei, H., G. Laporte, Y. Liu, and T. Zhang (2015). Dynamic design of sales territories. *Computers & Operations Research 56*, 84–92.

Lin, S. and B. W. Kernighan (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations research 21*(2), 498–516.

Merchán, D. and M. Winkenbach (2019). An empirical validation and data-driven extension of continuum approximation approaches for urban route distances. *Networks 73*(4), 418–433.

OpenStreetMap contributors (2025). OpenStreetMap. Accessed: 2025-04-25.

PostgreSQL Global Development Group (2025). PostgreSQL: The world's most advanced open source relational database. Version 17.5.

Rosvall, M. and C. T. Bergstrom (2008). Maps of random walks on complex networks reveal community structure. *Proceedings of the national academy of sciences 105*(4), 1118–1123.

Shapely (2025). Shapely.

Statista (2025). Global parcel shipping volume between 2013 and 2027 (in billion parcels)*.

9 Appendix

9.1 Listings

Listing 1: The algorithm to connect the buildings to the road network

```
for building in building_nodes:
    building_point: Point = building.point_lat_lon()
    nearest_index = tree.nearest(building_point)
    nearest_line: LineString = linestrings[nearest_index]
    nearest_edge: Edge = edge_map[nearest_line]
    start: Node = nearest_edge.start_node
    end: Node = nearest_edge.end_node

    projected_point: Point = nearest_line.interpolate(
        nearest_line.project(building_point)
    )
    projected_coords: tuple[float, float] = (
        projected_point.x,
        projected_point.y,
    )

    if projected_point.equals(Point((start.lat, start.lon))):
        new_edges.append(Edge(building, start))
        new_edges.append(Edge(start, building))
        continue
    if projected_point.equals(Point((end.lat, end.lon))):
        new_edges.append(Edge(building, end))
        new_edges.append(Edge(end, building))
        continue

    virtual_node_name: str =
        f"virtual_{next(node_id_counter)}"
    virtual_node: Node = Node(
```

```
            name=virtual_node_name,
            lat=float(projected_coords[0]),
            lon=float(projected_coords[1]),
            is_building=False,
        )
        new_nodes.append(virtual_node)

        self.delete_edge(nearest_edge)

        new_edges.append(Edge(start, virtual_node))
        new_edges.append(Edge(virtual_node, end))

        new_edges.append(Edge(building, virtual_node))
        new_edges.append(Edge(virtual_node, building))

    self.add_vertices(new_nodes)
    self.add_edges(new_edges)
```

9.2 Tables

Table 5: Empirical estimates for β, with prediction errors this beta gives for TSP path length in selected neighborhoods.

Province-Neighborhood	β	MAE (m)	MAPE (%)
groningen-Hortusbuurt	4.2726	787.0906	7.6233
groningen-Binnenstad	3.3907	944.3530	6.3492
groningen-Oosterpoort	3.6350	730.2659	7.2751
groningen-Rivierenbuurt	4.6877	747.5737	7.9876
groningen-De Wijert	2.6868	889.6834	6.1522
groningen-Oosterparkwijk	3.2167	1116.9408	6.4571
groningen-De Hoogte	3.1564	999.5812	9.1431
groningen-Korrewegwijk	2.9616	1080.9511	6.4441
groningen-Schildersbuurt	3.6631	729.9252	6.4071
groningen-Paddepoel	2.8494	729.3158	5.2808
groningen-Oranjewijk	3.2531	845.0706	6.8570
groningen-Tuinwijk	7.9827	1561.6483	14.7552
groningen-Selwerd	3.6898	782.9060	7.2691
groningen-Vinkhuizen	2.2486	743.4229	5.4375
groningen-Hoogkerk-zuid	2.6945	809.7548	6.3112
groningen-Gravenburg	2.5711	1681.3850	13.0166
groningen-De Held	4.2281	518.8655	4.4156
groningen-Reitdiep	3.0463	637.4352	5.1925
groningen-Hoornse Meer	2.9683	829.5638	7.2601
groningen-Corpus den Hoorn	3.5895	830.2959	6.7827
groningen-Eemspoort	3.5863	503.5335	4.0830
groningen-Euvelgunne	4.4215	1378.8606	9.7775
groningen-Driebond	4.1132	1604.2935	11.9177

Province-Neighborhood	β	MAE (m)	MAPE (%)
groningen-Winschoterdiep	5.0612	2655.1958	13.9624
groningen-Eemskanaal	3.5020	619.2529	4.2074
groningen-Helpman	3.7822	863.0136	6.7162
groningen-Lewenborg	3.7152	1289.0307	6.2147
groningen-Beijum	2.8381	737.1724	4.1504
groningen-Maarsveld	3.2871	602.9083	6.2278
drenthe-Assen Oost	2.2042	1138.7330	5.4455
drenthe-Assen West	2.2718	1965.7756	10.8618
drenthe-Centrum	2.6667	1245.6113	6.6355
drenthe-Kloosterveen	2.6103	2111.6730	8.8265
drenthe-Lariks	2.5490	1149.6956	7.7865
drenthe-Marsdijk	2.2432	1857.0997	7.1529
drenthe-Noorderpark	2.6897	837.7575	5.0655
drenthe-Peelo	2.7314	860.6995	6.0705
drenthe-Pittelo	2.9684	702.9646	5.4831
drenthe-Ter Borch	3.2532	699.8987	4.0286
friesland-Achter de Hoven	3.3134	621.8266	8.0718
friesland-Aldlân	2.3192	842.2286	5.3572
friesland-Bilgaard	2.8170	907.3997	6.9380
friesland-Binnenstad en Stationskwartier	3.7151	878.3273	6.4701
friesland-Blitsaerd	3.1162	831.4206	6.1789
friesland-Buitengebied Noordwest	2.7070	1581.5645	6.4575
friesland-Camminghaburen	2.5161	853.8815	4.8047
friesland-De Hemrik	2.9400	720.2788	4.4492
friesland-De Zuidlanden	3.0148	855.9684	6.0404
friesland-De Zwette	3.1604	1573.9905	6.6149
friesland-Heechterp	2.8824	824.5581	11.0612
friesland-Hempens, Teerns, en Zuiderburen	2.8145	1536.6074	6.3137
friesland-Huizum-Oost	3.2940	582.0815	4.8391
friesland-Huizum-West	2.5101	740.7885	6.4340
friesland-Middelsee	2.8683	872.5132	8.9713
friesland-Nijlân	2.6651	540.6227	5.3555
friesland-Oranjewijk	4.1201	580.7334	6.5334
friesland-Oud Oost	2.2604	778.7536	5.6172
friesland-Schepenbuurt	2.5741	1096.8162	15.6508
friesland-Schieringen en De Centrale	3.3044	923.4431	9.2577
friesland-Techum	3.6713	493.5220	5.2379
friesland-Vlietzone	2.5285	576.4998	5.8311
friesland-Vogelwijk en Componistenbuurt	3.2492	570.8451	6.5960
friesland-Vrijheidswijk	3.5769	499.8996	4.8038
friesland-Westeinde	3.2694	395.4296	3.6362
friesland-Wielenpôlle	2.9513	776.0476	10.8370
friesland-Zuiderburen	3.0958	1400.3054	5.8648
flevoland-Polderwijk	3.9953	1145.4509	5.9479
overijssel-Baalder	2.5008	717.4682	6.4571
overijssel-Baalderveld	2.4615	871.0699	6.2003
overijssel-Berflo Es	2.2532	1102.5143	6.3205

Province-Neighborhood	β	MAE (m)	MAPE (%)
overijssel-Bergweide	3.3208	928.4136	5.8873
overijssel-De Graven Es	2.9687	1471.4149	8.5628
overijssel-De Meijbree	3.3977	542.7416	6.8872
overijssel-De Riet	2.7811	684.2054	5.2536
overijssel-De Thij	2.3271	1031.6587	7.0723
overijssel-Groot Driene	2.5109	805.2004	4.2448
overijssel-Haardijk	4.2541	1097.9543	8.5854
overijssel-Hanzeland	3.6502	748.7525	8.7452
overijssel-Hasseler Es	2.3167	1101.6226	6.0418
overijssel-Heemsermars	3.5232	508.6826	6.1520
overijssel-Het Onderdijks	3.6251	447.5038	4.3605
overijssel-Hofkamp	2.7288	689.9057	5.3236
overijssel-Ittersum	2.2181	1267.4431	6.1395
overijssel-Keizerslanden	2.8282	1831.4711	9.0887
overijssel-Kloosterlanden	2.9544	973.5713	5.8996
overijssel-Marslanden	3.9237	1607.2791	9.0062
overijssel-Nieuwe Haven	4.2902	1317.1045	7.1460
overijssel-Nieuwstraatkwartier	2.9155	627.9651	6.4148
overijssel-Noorderkwartier	2.5561	787.4246	6.0342
overijssel-OosterDalfsen	4.6257	412.0292	5.6533
overijssel-Ossenkoppelerhoek	2.6807	835.1963	5.9376
overijssel-Rivierenwijk	2.9259	504.6057	4.8158
overijssel-Schelfhorst	2.6952	878.0725	4.8789
overijssel-Slangenbeek	2.1959	1138.7637	5.3182
overijssel-Sluitersveld	2.4855	1080.9257	6.7078
overijssel-Twekkelerveld	2.7282	755.1642	4.8439
overijssel-Veerallee	3.5561	806.9158	9.1483
overijssel-Voorstad	1.9748	883.2839	6.0335
overijssel-Wierdensehoek	2.8413	868.5459	5.8949
overijssel-Windmolenbroek	2.2649	1492.4184	5.8826
overijssel-Zandweerd	1.9880	819.2842	5.6229
noord holland-Schrijverswijk	3.2211	604.7393	5.5718
noord holland-Stad van de Zon	2.6047	2311.5860	15.9044
noord holland-Stadshart	4.7268	521.1349	4.8701
noord holland-Jordaan	2.9132	1214.6083	6.8566
noord holland-Slotervaart	2.3177	755.9461	4.8525
noord holland-IJburg	2.3454	898.3405	4.5361
noord holland-Oostelijke Eilanden	5.3696	623.7412	5.1655
noord holland-Oostelijk Havengebied	3.5957	1045.0941	4.7407
noord holland-Frederik Hendrikbuurt	5.1406	1038.7155	8.9046
noord holland-Van Lennepbuurt	4.1208	768.0134	6.5499
noord holland-Da Costabuurt	9.4503	930.6030	8.6875
noord holland-Kinkerbuurt	8.8381	998.1282	8.7462
noord holland-Kersenboogerd	2.4016	1145.6057	5.2513
noord holland-Pax	3.6171	562.4764	3.9765
noord holland-Graan voor Visch	3.4760	547.0381	5.5134
noord holland-Vrijschot-Noord	4.6419	710.2193	8.0498

Province-Neighborhood	β	MAE (m)	MAPE (%)
noord holland-Toolenburg	2.6987	987.3229	4.3092
noord holland-Floriande	2.7583	1117.5578	4.3851
noord holland-Overbos	2.4249	651.6643	4.0632
noord holland-Bornholm	2.9670	642.8920	4.3094
noord holland-Beukenhorst-Oost	4.4004	1040.6298	8.1963
noord holland-De Hoek	5.1934	1003.2119	6.2698
noord holland-West	3.3529	446.6162	5.8976
noord holland-Zuid	4.3001	903.2890	5.8866
noord holland-Oost	3.3538	766.3976	5.2967
noord holland-Noord	2.9332	706.6457	5.5368
noord holland-De President	2.7572	1122.8178	11.4476
noord holland-Graan voor Visch-Zuid	3.8440	687.4784	6.8385
noord holland-Zuidwijk	3.4466	456.7274	4.0636
noord holland-Buitenveldert-West	2.6663	1218.8317	6.4754
noord holland-Buitenveldert	2.0304	1492.3560	6.9382
noord holland-Apollobuurt	3.0360	933.7595	7.0048
noord holland-Stadionbuurt	3.0005	972.0317	7.9713
noord holland-Prinses Irenebuurt e.o.	4.8210	578.6442	6.8676
noord holland-Hoofddorppleinbuurt	3.0742	1169.3999	8.5072
noord holland-Willemspark	5.1500	1013.1195	8.3730
noord holland-Schinkelbuurt	7.9243	1289.2277	11.4147
noord holland-Vondelparkbuurt	5.6385	855.1568	8.8287
noord holland-Helmersbuurt	4.2320	814.9797	6.9407
noord holland-Overtoomse Sluis	7.6007	746.8328	6.8497
noord holland-Museumkwartier	3.0306	1145.3162	7.3443
noord holland-Rivierenbuurt	2.6298	1242.1375	6.8000
noord holland-IJselbuurt	4.9946	1027.3968	8.9081
noord holland-Scheldebuurt	4.0386	970.1862	6.5056
noord holland-Rijnbuurt	5.5333	593.4113	5.2244
noord holland-De Baarsjes	2.7382	1255.6146	6.0825
noord holland-Landlust	4.1666	1084.6529	7.1576
noord holland-Staatsliedenbuurt	3.1675	815.2870	7.0086
noord holland-Spaarndammerbuurt	5.1401	811.4117	7.0284
noord holland-De Pijp	2.9061	1361.7288	6.8957
noord holland-Grachtengordel	3.2421	1425.1466	7.2483
noord holland-Oud-Zuid	1.9600	1461.1687	5.7338
utrecht-Bedrijventerrein Vathorst	4.7281	952.7612	5.2408
utrecht-Binnenstad City- En Winkelgebied	4.4268	687.2519	6.5254
utrecht-Binnenstad Woongebied	3.8434	1100.1967	5.6842
utrecht-Bosgebied	3.0752	1367.0046	6.3416
utrecht-Buitengebied Oost	2.6066	1397.7654	6.7521
utrecht-Calveen	4.0031	899.3645	8.5463
utrecht-De Berg-Noord	2.8072	821.9696	5.0387
utrecht-De Berg-Zuid	2.6803	932.6851	6.0254
utrecht-De Hoef	3.3696	1110.2511	8.6874
utrecht-De Koppel	3.7019	476.2108	5.2298
utrecht-Dichterswijk, Rivierenwijk	2.3669	787.9536	5.2042

Province-Neighborhood	β	MAE (m)	MAPE (%)
utrecht-Eemkwartier	4.4390	695.4076	7.9055
utrecht-Hoogland	2.6734	1054.4349	5.7404
utrecht-Hoogland-West	2.1920	1942.9992	7.0928
utrecht-Hooglanderveen	3.6108	676.3181	4.4060
utrecht-Isselt	3.5409	999.1430	5.5305
utrecht-Kanaleneiland	2.4141	684.5170	4.6111
utrecht-Kattenbroek	2.9860	840.8940	5.0859
utrecht-Kruiskamp	3.3966	778.2176	6.4218
utrecht-Leusderkwartier	2.3377	1041.8890	7.3030
utrecht-Liendert	3.0075	590.8835	4.6034
utrecht-Lombok, Leidseweg	2.7542	848.2440	5.8439
utrecht-Nederberg	5.3111	731.6925	6.7624
utrecht-Nieuw Engeland, Schepenbuurt	2.8475	2120.2284	6.9265
utrecht-Nieuwland	3.0729	838.1784	4.7925
utrecht-Oog in Al	2.8049	1044.5446	7.2653
utrecht-Randenbroek	2.9912	782.4703	5.3747
utrecht-Rustenburg	2.8153	395.7474	3.9862
utrecht-Schothorst-Noord	2.4407	954.6565	6.7492
utrecht-Schothorst-Zuid	3.6148	649.1111	4.5797
utrecht-Schuilenburg	3.0351	552.0899	5.6674
utrecht-Soesterkwartier	2.7149	1101.6736	6.9427
utrecht-Stadskern	4.2535	865.9577	5.9803
utrecht-Terwijde	3.1874	735.9790	3.9313
utrecht-Vathorst-Centrum	4.1298	774.4471	6.0928
utrecht-Vathorst-De Laak	2.5895	1214.6085	5.9675
utrecht-Vathorst-De Velden	3.1590	737.7602	4.2696
utrecht-Veldhuizen	2.8434	714.6507	5.2290
utrecht-Vermeerkwartier	2.4646	696.3630	5.2603
utrecht-Zielhorst	2.9019	833.9362	6.3967
gelderland-Aldenhof	3.8426	556.8623	5.8121
gelderland-Altrade	3.2871	783.9319	6.2079
gelderland-Benedenstad	3.8464	390.0724	4.3911
gelderland-Biezen	3.2305	827.2408	5.6937
gelderland-Bijsterhuizen	4.8726	1566.6764	12.7655
gelderland-Bottendaal	4.2064	659.9065	6.9027
gelderland-Brakkenstein	2.2316	754.9688	6.7204
gelderland-De Hoop	3.1835	522.6107	5.6437
gelderland-De Horsten	3.3369	566.2992	7.1855
gelderland-De Huet	2.3466	629.8263	3.9225
gelderland-De Kamp	3.7104	959.6935	6.1985
gelderland-De Pas	3.4322	692.0413	7.0011
gelderland-Dichteren	3.0613	1103.4474	7.4356
gelderland-Druten-Zuid	2.2404	765.5168	4.9191
gelderland-Ede-Zuid	2.3679	862.7631	6.0182
gelderland-Enka	3.2879	497.8380	4.9358
gelderland-Galgenveld	3.2705	942.6533	6.8689
gelderland-Goffert	2.3635	977.5300	5.7219

Province-Neighborhood	β	MAE (m)	MAPE (%)
gelderland-Groenewoud	2.8702	633.9448	4.6512
gelderland-Grootstal	3.0195	752.7934	5.8744
gelderland-Hatert	2.6656	792.0939	5.0487
gelderland-Haven- en industrieterrein	3.2732	1524.4291	8.3785
gelderland-Hazenkamp	2.7951	822.3193	5.7842
gelderland-Hees	2.9247	556.8466	4.3939
gelderland-Heideslag	3.2618	925.1216	11.0459
gelderland-Heijendaal	2.9010	1183.4906	8.4884
gelderland-Hengstdal	3.1750	626.3845	4.6735
gelderland-Heseveld	3.4145	987.7851	6.6970
gelderland-Hunnerberg	3.0523	1043.7581	7.4868
gelderland-Kerkenbos	3.6606	931.5513	9.1516
gelderland-Kernhem	2.6408	1654.5925	9.7877
gelderland-Kortenoord	3.6543	406.3872	3.6853
gelderland-Kwakkenberg	2.4521	883.1879	6.4522
gelderland-Lankforst	4.3673	420.6216	4.6022
gelderland-Lent	2.6871	1670.4846	6.7304
gelderland-Maandereng	2.8238	569.2160	4.9578
gelderland-Malvert	4.4647	517.8930	5.0788
gelderland-Meijhorst	4.6960	523.7948	4.3021
gelderland-Methen	3.0645	385.6225	3.3874
gelderland-Neerbosch-Oost	3.3565	1083.8216	6.9023
gelderland-Nije Veld	2.8840	690.8362	5.3239
gelderland-Noordwest	3.0003	500.2537	4.2772
gelderland-Nude	2.7880	521.1733	4.5426
gelderland-Oosseld	2.6560	721.2955	5.7656
gelderland-Oosterhout	3.0951	907.2217	5.2443
gelderland-Ooyse Schependom	4.3891	541.3111	9.9971
gelderland-Overstegen	3.2436	538.8126	3.6453
gelderland-Ressen	2.6004	2499.0589	15.6158
gelderland-Rietkampen	3.5870	568.7235	3.7161
gelderland-Rijkerswoerd	2.4182	768.0688	4.4788
gelderland-Roodwilligen	3.3730	214.7114	5.4052
gelderland-Schöneveld	3.0271	512.8591	5.2261
gelderland-Staddijk	3.9389	970.2252	5.1446
gelderland-Stadscentrum	3.3501	1043.4735	7.1672
gelderland-Stadsweiden	2.5235	502.7610	3.7071
gelderland-Tolhuis	3.8805	1161.7093	8.3833
gelderland-Veldhuizen	2.4465	673.1475	4.1493
gelderland-Weezenhof	3.7573	637.6382	4.4508
gelderland-Westkanaaldijk	3.9640	911.8447	5.6946
gelderland-Wijnbergen	3.5748	382.2284	4.4317
gelderland-Wolfskuil	3.2695	801.0343	6.4680
gelderland-Zwanenveld	3.7899	454.3413	3.7634
zuid holland-Afrikaanderwijk	5.1813	755.9314	6.4506
zuid holland-Berkel	2.9210	732.0691	5.5160
zuid holland-Beverwaard	1.9625	740.6548	5.6274

Province-Neighborhood	β	MAE (m)	MAPE (%)
zuid holland-Binnenstad	2.5989	1091.9128	6.1208
zuid holland-Bloemendaal	2.3656	824.0422	4.3516
zuid holland-Bloemhof	3.7410	1095.6418	7.3129
zuid holland-Bospolder	3.5109	878.6328	7.5607
zuid holland-Buitenhof	3.0559	1278.5928	7.7260
zuid holland-Capelle-West	3.0559	736.4316	7.1468
zuid holland-Carnisse	4.0828	983.6190	7.5457
zuid holland-Cool	4.8862	1248.6295	8.0823
zuid holland-De Schenkel	3.2953	543.5715	4.9991
zuid holland-Delfshaven	5.0730	981.4805	7.7313
zuid holland-Fascinatio	2.8522	1086.6211	9.9432
zuid holland-Feijenoord	4.1307	668.7462	5.7364
zuid holland-Goverwelle	2.7867	850.3882	6.2137
zuid holland-Groente- en Fruitmarkt	4.1215	667.9367	6.3517
zuid holland-Groot-IJsselmonde	2.3024	1385.9490	5.7558
zuid holland-Heijplaat	4.1048	696.5381	7.5029
zuid holland-Hellevoet	3.2695	1068.7121	6.3925
zuid holland-Het Lage Land	3.0240	965.0769	6.3200
zuid holland-Hillegersberg-Noord	2.6365	1412.1092	8.7149
zuid holland-Hillesluis	3.8656	1022.6123	6.9756
zuid holland-Hof van Delft	2.2735	805.8094	5.8004
zuid holland-Hoog Dalem	3.4868	463.4984	3.8859
zuid holland-Katendrecht	4.0872	974.0655	7.8884
zuid holland-Kleinpolder	3.3590	664.1459	5.0661
zuid holland-Kleiwegkwartier	3.0073	1105.2103	7.7506
zuid holland-Kop van Zuid-Entrepot	4.2950	1013.3788	7.3530
zuid holland-Kort Haarlem	2.7007	799.0229	5.5994
zuid holland-Korte Akkeren	2.1746	1048.7966	7.4106
zuid holland-Leyenburg	3.3060	1182.8900	7.1212
zuid holland-Lombardijen	2.9791	1012.2313	6.0138
zuid holland-Middelland	4.9408	1405.2185	8.7951
zuid holland-Middelwatering	2.5498	1199.0042	6.0586
zuid holland-Moerwijk	2.7064	1037.6902	5.6828
zuid holland-Molenlaankwartier	2.1717	1289.8516	7.8659
zuid holland-Morgenstond	3.0459	1169.2886	5.8592
zuid holland-Nesselande	2.7498	1201.7011	5.3897
zuid holland-Nieuw-Helvoet	3.0116	892.1968	5.9277
zuid holland-Nieuw-Mathenesse	4.4945	2485.7026	12.1612
zuid holland-Nieuwe Westen	2.7293	1053.2857	6.8357
zuid holland-Noord	2.3743	1034.7771	6.0713
zuid holland-Noordereiland	5.3619	623.5225	6.9333
zuid holland-Ommoord	2.8623	1142.8971	4.7820
zuid holland-Onyx	3.9427	351.0077	6.8568
zuid holland-Oosterflank	2.8279	986.9776	6.4671
zuid holland-Oostgaarde	2.6475	1008.7880	5.7550
zuid holland-Oud-Charlois	2.8713	991.6265	6.1135
zuid holland-Oud-IJsselmonde	3.0651	1100.4647	5.5309

Province-Neighborhood	β	MAE (m)	MAPE (%)
zuid holland-Oud-Mathenesse	3.8736	792.9639	6.9276
zuid holland-Oude Westen	5.0152	1300.9461	8.6358
zuid holland-Overschie	2.8572	932.9413	6.1333
zuid holland-Pendrecht	3.4695	841.9458	6.3492
zuid holland-Plaswijck	2.7140	1000.2560	5.7854
zuid holland-Portlandsehoek	3.2111	533.4824	5.3000
zuid holland-Prinsenland	3.3168	676.7650	4.1853
zuid holland-RijswijkBuiten	3.4631	827.7648	5.3352
zuid holland-Rivium	3.8937	578.3450	6.4807
zuid holland-Ruiven	2.3846	1168.6130	8.7792
zuid holland-Rustenburg en Oostbroek	2.7888	1048.4608	6.4569
zuid holland-Schenkel	2.8453	705.7863	5.2924
zuid holland-Schiebroek	2.0109	1519.2914	7.7388
zuid holland-Schieweg	2.6786	1185.2443	8.3900
zuid holland-Schollevaar	2.8953	2209.5557	9.1045
zuid holland-Spangen	4.5173	888.4093	6.9012
zuid holland-Stadsdriehoek	3.3578	1132.8572	5.9851
zuid holland-Stolersluis	3.4987	918.4346	10.0803
zuid holland-Tanthof-Oost	2.9945	538.2743	4.8544
zuid holland-Tanthof-West	3.1704	750.5216	6.1426
zuid holland-Tarwewijk	2.9423	1128.1956	7.8602
zuid holland-Transvaalkwartier	3.3665	1115.0289	7.2378
zuid holland-Tussendijken	6.0265	1038.8885	8.7457
zuid holland-Vogelbuurt	3.3138	379.0538	4.4785
zuid holland-Voordijkshoorn	3.3601	1023.5672	5.1693
zuid holland-Voorhof	3.5082	755.0064	5.6075
zuid holland-Vreewijk	2.6371	1340.8887	6.5759
zuid holland-Vrijenban	2.3398	1052.3212	7.2528
zuid holland-Wateringse Veld	2.2992	1024.2319	4.4440
zuid holland-Westergouwe	3.0444	1447.7450	8.5294
zuid holland-Westplaat	2.8120	495.1839	5.1190
zuid holland-Westpolder	3.8212	1222.4980	7.3129
zuid holland-Wielewaal	4.6079	673.9397	8.1491
zuid holland-Wippolder	2.4995	839.7443	5.0233
zuid holland-Zestienhoven	2.7034	1440.4121	6.3112
zuid holland-Zevenkamp	2.5450	1007.0375	5.9420
zuid holland-Zuid	3.3443	447.6298	5.1557
zuid holland-Zuiderpark	4.1519	814.2675	7.4086
zuid holland-Zuidwijk	2.4875	800.1075	5.8828
noord brabant-Binnenstad	2.1495	1061.1476	5.4125
noord brabant-Brandevoort	2.2347	1839.7531	9.5962
noord brabant-Brouwhuis	2.1358	1609.2903	8.9486
noord brabant-Dierdonk	2.2002	2250.6841	13.1282
noord brabant-Helmond-Noord	2.4727	855.0128	4.8511
noord brabant-Helmond-Oost	2.4375	845.5918	6.0105
noord brabant-Helmond-West	2.8577	914.0061	7.2283
noord brabant-Industriegebied Zuid	2.4041	1882.7544	6.9277

Province-Neighborhood	β	MAE (m)	MAPE (%)
noord brabant-Mierlo-Hout	2.2206	858.5567	4.3769
noord brabant-Oosterheide	2.2711	1295.6620	7.2752
noord brabant-Rijpelberg	2.1845	3279.7709	18.9678
noord brabant-Stiphout	1.9437	1479.6214	6.8681
noord brabant-Warande	2.7189	881.0406	4.7756
noord brabant-West	2.2574	2131.3435	7.8855
noord brabant-Zuidoost	1.9708	2389.1777	11.3801
limburg-Blerick-Midden	2.8979	775.1008	5.9964
limburg-Blerick-Noord	3.1005	768.3499	6.2648
limburg-Blerick-Zuid	3.2060	468.5901	5.0028
limburg-Boekend	2.3537	1603.2939	11.3346
limburg-Centrum	2.3576	1642.7264	7.0409
limburg-Donderberg	2.6024	577.4141	4.5257
limburg-Hoogvonderen	3.0845	379.7682	3.7412
limburg-Hout-Blerick	2.1900	1632.7321	9.6315
limburg-Klingerberg	2.7365	613.5472	6.3093
limburg-Lindenheuvel	2.2620	1057.2441	6.1935
limburg-Maasniel	2.2197	1445.4235	7.2838
limburg-Maastricht-Centrum	3.3288	1329.2311	5.8712
limburg-Maastricht-Noordwest	3.0218	998.9314	6.5367
limburg-Maastricht-Oost	1.9199	1334.7535	4.7881
limburg-Maastricht-West	1.9968	1194.1614	4.4141
limburg-Maastricht-Zuidoost	1.7929	1189.8679	5.0765
limburg-Meuleveld	3.9232	665.7765	6.9355
limburg-Roermond-Oost	2.1133	743.0770	5.6894
limburg-Roermond-Zuid	1.9483	2222.7765	11.9935
limburg-Sanderbout	3.4633	793.4146	8.2930
limburg-Vossener	3.1131	541.2249	5.7338
zeeland-Binnenstad	2.4821	1169.1696	7.9141
zeeland-Burgh	3.2653	647.7810	7.3633
zeeland-Haamstede	3.2263	647.5954	4.9538
zeeland-Lammerenburg	2.3572	1272.1640	5.7762
zeeland-Middelburg-Zuid	3.0436	1052.4194	7.1346
zeeland-Othene	3.1413	861.1507	5.6943
zeeland-Paauwenburg	2.2499	1093.2731	7.0659
zeeland-Terneuzen-Centrum	3.6704	668.4812	5.7383
zeeland-Terneuzen-Noord	2.7357	804.5580	4.9723
zeeland-Terneuzen-West	2.3632	1023.0001	6.7493
zeeland-Terneuzen-Zuid	2.4129	653.2935	4.1900
zeeland-West-Souburg	3.2091	786.4329	6.8031

Table 6: Per-area MAPE from Random Forest models using all features and reduced features.

Province-Neighborhood	MAPE (%) (F)	MAPE (%) (R)
drenthe-Centrum	10.05	8.13
drenthe-Ter Borch	6.45	5.88

Province-Neighborhood	MAPE (%) (F)	MAPE (%) (R)
friesland-Bilgaard	7.89	7.95
friesland-De Hemrik	5.75	6.40
friesland-De Zuidlanden	24.45	19.22
friesland-Huizum-Oost	6.81	10.28
friesland-Huizum-West	7.13	7.52
friesland-Techum	13.80	13.70
gelderland-Aldenhof	12.30	10.04
gelderland-Altrade	7.31	5.63
gelderland-Bijsterhuizen	24.25	19.48
gelderland-Dichteren	13.59	10.20
gelderland-Heideslag	12.45	11.30
gelderland-Lent	10.71	8.96
gelderland-Malvert	15.30	13.82
gelderland-Nude	10.10	11.09
gelderland-Rietkampen	12.12	4.44
gelderland-Stadscentrum	7.05	7.04
gelderland-Weezenhof	10.79	10.78
gelderland-Wolfskuil	5.63	5.43
groningen-Maarsveld	12.98	10.17
groningen-Oosterpoort	11.13	7.70
groningen-Vinkhuizen	11.29	10.41
limburg-Klingerberg	9.48	6.26
limburg-Maastricht-Oost	5.80	6.49
limburg-Roermond-Oost	7.19	5.90
noord brabant-Brouwhuis	9.11	9.77
noord brabant-Stiphout	18.73	21.41
noord brabant-Zuidoost	18.40	20.50
noord holland-Buitenveldert-West	6.79	6.19
noord holland-De Hoek	22.11	27.46
noord holland-Floriande	12.67	17.41
noord holland-Jordaan	8.84	8.33
noord holland-Kersenboogerd	6.03	7.07
noord holland-Museumkwartier	7.68	7.00
noord holland-Noord	6.06	5.24
noord holland-Scheldebuurt	6.32	6.12
noord holland-Slotervaart	4.90	4.98
noord holland-Staatsliedenbuurt	6.69	8.74
noord holland-Stad van de Zon	23.92	21.99
noord holland-Stadionbuurt	6.76	7.48
noord holland-Vondelparkbuurt	17.98	9.72
noord holland-Vrijschot-Noord	12.24	15.47
overijssel-De Meijbree	6.22	7.38
overijssel-De Riet	5.04	5.94
overijssel-Nieuwstraatkwartier	5.73	5.81
overijssel-Rivierenwijk	6.96	6.77
overijssel-Voorstad	32.11	31.69
overijssel-Windmolenbroek	8.29	10.20

Province-Neighborhood	MAPE (%) (F)	MAPE (%) (R)
utrecht-Binnenstad City- En Winkelgebied	8.29	12.69
utrecht-Calveen	16.96	19.90
utrecht-De Berg-Zuid	6.69	6.90
utrecht-Dichterswijk, Rivierenwijk	5.63	4.99
utrecht-Kattenbroek	12.19	16.45
utrecht-Liendert	5.64	6.48
utrecht-Nieuwland	6.20	6.50
utrecht-Randenbroek	5.94	5.63
utrecht-Terwijde	5.62	7.81
utrecht-Zielhorst	7.08	6.67
zeeland-Terneuzen-West	10.50	9.74
zuid holland-Bospolder	7.42	7.78
zuid holland-Cool	15.34	17.08
zuid holland-Fascinatio	19.29	14.17
zuid holland-Groente- en Fruitmarkt	29.71	33.15
zuid holland-Heijplaat	8.08	7.02
zuid holland-Het Lage Land	8.08	7.56
zuid holland-Hillesluis	11.01	10.53
zuid holland-Hof van Delft	8.82	9.54
zuid holland-Hoog Dalem	5.52	5.68
zuid holland-Katendrecht	11.59	7.22
zuid holland-Kleiwegkwartier	8.16	6.48
zuid holland-Morgenstond	10.40	8.35
zuid holland-Nieuw-Helvoet	5.86	7.33
zuid holland-Rustenburg en Oostbroek	8.59	6.56
zuid holland-Schiebroek	11.26	13.21
zuid holland-Schollevaar	13.89	16.15
zuid holland-Stolersluis	26.79	24.70
zuid holland-Wippolder	8.44	6.82
zuid holland-Zevenkamp	7.76	5.53
Total	10.86	10.58

10 Use of AI

I made use of AI-based tools during the course of this research, primarily ChatGPT, but also sometimes GitHub Copilot or DeepSeek when the daily limit of the GPT-4 model in ChatGPT was reached. These tools were used for the following purposes:

- At the start of the project, I used these tools to help come up with ideas and approaches. Most of the suggestions were either incorrect or inefficient. However, this can still be useful since this makes you think critically about the problems at hand. If you understand exactly what is wrong or inefficient about the idea that an AI tool generated, this knowledge can be used to come up with a better idea.
- For code-related tasks, I used these tools to assist with writing syntax I find annoying to write myself, such as making plots. I noticed during the project that AI is not useful for writing key parts of the programs, the code it generates is almost always

wrong. A text editor with a good LSP (Language Server Protocol) (like `pyright` for `Python`) is much more useful in providing auto-completion and error-checking features for programming, compared to AI tools.

- For writing and editing text, these tools were occasionally used to help draft or rephrase sections of this report, particularly to improve clarity or flow.